Code Smarter With

SpectronX

Developed By:
Ronaldo Excellent

Table of Contents

- **Table of Contents 2**
- **SPECTRONX (Full Intro) 4**
- **More Info on SpectronX 7**
- **Package.cjs 11**
- **Index.html 12**
- **Root.css 12**
- **App.cjs 12**
- **Library.cjs 12**
- **The App Object 13**
- **Selectors 14**
- **ROM Components 14**
- **SpectronX Date 15**
- **SpectronX Constructors 15**
  - **Layout Constructor 16**
  - **Widgets Construction & Rendering 16**
- **Other SpectronX Functions 17**
- **Widget Cloning 18**
- **Content Manipulation 18**
  - **Contenting 18**
  - **Styling 19**
  - **Functionality 19**
  - **Animation 20**
  - **Visibility State 20**
  - **Iteration 20**
  - **Integers 21**
  - **Shuffle, Remix & Typewrite 22**
  - **Tracing & Untracing Elements 22**
  - **Switch between Contents 23**
  - **Popping Widgets 23**
  - **CSS Transform Functions 23**

Binding data to Iframe.....24
Manipulating Classes 24
Audio Callbacks (For Chrome and Edge Browsers).....25
- **Events Handling.....25**
- **Objects 25**

SPECTRONX (Full Intro)

History

In the year 2022, I upgraded my ability to solve web frontend problems and even wrote my first web game (https://spellingbee1.vercel.app). Then I made few other games, because making websites was a bit boring. Then making more apps and games no longer became a major objective. I was still unsatisfied and wanted to do more than just writing apps and games. I thought of what I could create as a programmer that would help/benefit other programmers, then I thought of creating a very little library. I started having real visions of building it, writing real time problem solving pieces of code. After a few months, I decided to remake it and make it more complex, consisting of functions and some extra features that lets you bind and call application data and even external variables whenever and wherever you need them. I also added some special code to help solve some special problems, especially in the area of design and functionality. The former (nano-sized) library, now a full frontend development framework is finally ready.

What is SpectronX?

SpectronX is a JavaScript Frontend Development Framework that focuses on HTML DOM and lets one manipulate it easier and faster. With SpectronX, you can;

- Build & manipulate web contents easier & faster.
- Bind app and Meta data with less lines of code.
- Wrap new styles for objects in a single JS function.
- Directly insert variables from an external JS file.
- Create single page applications.
- Manipulate contents faster.

- Build your web based (puzzle) games easier & faster…

CONCEPT

SpectronX is a framework for web frontend development that was built to solve complex problems easier and faster, while enabling the developer or programmer to write less code on frontend languages like HTML5, CSS3 & JavaScript.

SpectronX was built to solve web frontend complex problems.

It also has a CSS framework and so many JavaScript functions for designing/programming web pages and web apps faster.

There are 4 major ways to use SpectronX

- **General Page Design & Manipulation:** This involves the normal way to handle web design by the proper combination of;

HTML - Proper layout and content creation.

CSS - Proper styling (with framework support).

JavaScript - Proper content and style manipulation.

CREATING CUSTOM COMPONENTS

- **Standard HTML Components**: You can create and render your own components in standard HTML format, using the components object in (package.cjs). This components can be used in all pages once they're add to them using the <script> tag/element. You can either write them on single lines or as arrays. For example, if you wanna create a component called <mybtn> as a button, go to package.cjs in your root folder and add the element "mybtn" to the elements array, then go to components and "mybtn" as an object property to the components object variable. Then add this data to it: <attrib style="background: green; color: white; font-weight: 800;"></attrib> Click me!

Now open the file you linked the package.cjs to (if you haven't done it, make sure you do), in between your layout element, add <mybtn></mybtn> then preview the webpage on your browser to see the component you just created. The <attrib> element is used to add attributes to your components. They may include "class", "id", "style", or even "onclick".

- **Building Components With JavaScript:** These are components created and rendered with SpectronX JS functions.

Here's an example of component creation with JS:

```
// Check if mybtn element exists
if ($("mybtn")) {
	// Loop through all existing mybtn elements, necessary if you
	wanna use the component multiple times on a single page.
	loopX($$("mybtn"), () => {
		// Add HTML & CSS
		html(i, "Click me"), css(i, "background: blue; color:
		white;);
		// You might decide to use this instead of css...
		addClasses(i, ["s-blue", "s-color-white", "s-padding", "s-
		text-large");
	);
}
```

That's it! You can now add the <mybtn> element anywhere on your webpage if it's connected with the script, then save and preview to see results.

Requirements

To use SpectronX, you must have a basic knowledge and understanding in;

- **HTML5** - Standard Markup Language for structuring, and defining the layout of a webpage/web app.

- **CSS 3** – Used for styling a webpage/web app.

- **JavaScript** - A programming language, basically for programming the behaviour of a webpage/web app.

More Info on SpectronX

SpectronX is a framework that tests & proves how smart & capable people are in solving web frontend problems in design and functionality, while providing them with tools and functions to help them code faster on a large scale.

WORKING WITH SPECTRONX

Let's talk about working with SpectronX as a developer. As you might have noticed, SpectronX is a framework that contains that is built upon multiple files. To be able to work well with this tool, you must know some or most of these files. For example:

UNDERSTANDING DATA INTERCHANGE & BINDING WITH SPECTRONX FRAME

SpectronX lets you bind your major web app data using the <**application />** element. This element binds our application data and contain the three main elements for our app development.

The following are attributes of the **<application />** element:

logo – This stores your application/website logo src and places it where it's needed. As we continue the journey, you'll understand fully.

The rest are located and should be filled in package.cjs file.

name – This holds the name of the application/website and sends it to the head element under the title element and application-name attribute in the meta tag/element.

author – This stores the name of the author/company that owns the app.

theme – This is used to specify a theme (major colour) for your application.

mode – This is used to specify a theme between (light or dark) for the entire app.

Projectstart – This holds the date/time of when your project development began.

Projectend – This holds the date/time of when you finished your project.

Sample:

```
<application logo=""></application>
```

There are three sections in the application element that contains our entire (visible) content. We have the;

<data></data> section -- For containing our meta data (& other valuable data if need be).

<layout></layout> section -- For containing our HTML code.

<engine></engine> section -- For containing our external scripts (codejs.cjs & app.js).

As you can see, all meta tags are contained with the **<data>** element, and are binded in three lines. One for the **<meta charset>**, then the **<meta-name>** (instead of **<meta name="" content="">**, and the **<meta-http-equiv>** (instead of **<meta httpequiv>**), as shown below;

```
<data>
    <meta charset="charset">
    <meta-name name="content">
    <meta-http-equiv name="content">
 </data>
```

The **<layout />** element contains our main content as shown below;

```
<layout>
    <header></header>
    <nav></nav>
    <main>
        <section>
            <article></article>
        </section>
    </main>
    <aside></aside>
    <footer></footer>
</layout>
        </_Comments goes here>
```

And comments in HTML can also be written as;

```
        </_Comments goes here>
```

Then we have the <engine></engine> element for containing our scripts, including adding from other sources.

Note:

All existing files in the template have been completely linked/wired together, so do not link again to avoid unnecessary errors. Goto css/stylesheets.css to import CSS stylesheets or libraries, even

without adding <link /> to the index.html, but in case, you can use the <link /> in <data> element, after the <meta-http-equiv /> element.

You can insert all app data using "js" attribute, and <js> element on any of your pages if linked to package.cjs & library.cjs. Use back tick/back quote to join sentences with app object data.

```
<layout>
        <js>app.logo</js>
        <h1 js>`You're welcome to ${app.name}`</h1>
        </_Inserting math calculations>
        <js> (2+2) * 4 – Math.sin(2) / 80 * 200 </js>
        <js>` I am 50 * 50 years old `</js>
</layout>
```

Then Check the result! You can check your console to confirm the new comment tag (</_Comment>).

You can also run and display math calculations either using JS element & js attribute. E.g.

Package.cjs

To start using a SpectronX package, go straight to this file and fill the details, the app name, author (the owner of the website), the

mode (light/dark theme), theme, the date when the project began (projectstart) and ended(projectend) (*optional*). These can be called using the app object. You can also use it to create your web components (reusable code) in HTML standard format using the components object already existing within the file, by first add the name of a custom element with the elements array, then adding the same element (name) in the components object and adding the value in HTML standard format then including it/them in any webpage that is linked with the package.cjs file as you would link a JS script to an HTML page..

Index.html

This will content the first contents (like a homepage) that you want your visitors to see.

Root.css

This is the main CSS file for the entire website/web app. Link all your pages with it to use the CSS library for quick design. You can also use it to write CSS for your index file.

App.cjs

This is the external script for your index file.

Library.cjs

This is the main JavaScript/CJS file for the entire website/web app. Link all your pages with it to use its features which we'll discuss later in this ebook.

Notes:

On every page, always like package.cjs first, then library.cjs from the source folder located within the js folder, then your external script if you created any, or internal script if you want.

To see the version of SpectronX that you're currently using, write **spectronX.*version*** and you'll get it.

The App Object

app.name – This returns the application's name.

app.author – This returns the application's author.

app.logo – This returns the logo of the app as linked through the logo attribute in the application element within all your pages.

app.theme – This returns the current theme of the app.

app.mode – This returns the current mode of the app.

app.footnote() – This function generates a footnote to your web app.

app.timer.clone(targ) – This function lets you clone the app timer (program that tells you how long you've spent on a website or web app, located in the app console (different from browser console).

app.log(data) – This function lets you add send errors to the app console.

app.state(state) – This app lets you change the state of your website/web app. The state could be under

development, repair, maintenance, upgrade or even temporarily shutdown.

app.messages.show() – This function lets you display the app console.

app.messages.hide() - This function lets you hide the app console.

Note: These functions also work well with buttons when inserted using the onclick or click() function which we'll be discussing in this ebook.

Selectors

The following are used to select HTML elements in SpectronX;

$(element_name) – This is used to select single elements.

$$(element_name) - This is used to select a group of elements.

Note: Both work just like CSS selectors, that is they let you use # for ID, "." for class (class names) and "[ ]" for attributes and attribute values. The group selector works with SpectronX loop functions, which make iteration for both arrays, objects and HTML elements easier and faster.

ROM Components

SpectronX ROM Components are components containing special data for special purposes and which are called upon wherever necessary. This components have already been created and stored in SpectronX and their sole aims/purposes are for the advancement and development of websites / web apps.

There are two (2) types of ROM Components in SpectronX.

i) **Single ROM Components:** These are components that were built with only JavaScript.

E.G: Date components...

ii) **Combined ROM Components:** These are components that were built by combining JavaScript and other languages like HTML5 & CSS3.

E.G: SpectronX Objects (*Will be introduced later)...*

SpectronX Date

This object is used to insert date to your web page / web app. In your layout element in the index.html file, write:

<js> d.fullDate() </js>

And preview to see the full date according to device's system's time.

You can also write to your app.js file:
html($('layout'), d.fullDate()); It'll

still give the same results.

Here's a full list of all the date functions:

d.day()

d.date()

d.month()

d.year()

d.fullDate()

SpectronX Constructors

If you want to build/design and program a web app with JavaScript, SpectronX Builder (building functions) will help you accomplish it easier and faster.

Layout Constructor

If your website contains one header, navigation (nav), main (for the major contents including sections and articles), aside & footer, simply use the **layout.create()** function to create all at once, e.g: ***layout.create();***

And, use **layout.destroy()** function to complete empty/destroy the layout (contents.

Note: You can check your console to see messages.

Widgets Construction & Rendering

In SpectronX, HTML elements are treated as widgets and are given the default class "widget" when created.

To create an element, use the widgets.construct() function.

Syntax: widgets.construct(element)

To add the element to your layout, use the render() function.

Syntax: render(element, target) => render(widget, $('layout'));

To create multiple elements at once, use widget.constructMultiple() and use renderAll() to generate all at once.

Note: You can also use any of the render() functions to move an already existing element from one place to another, on the web page.

Other SpectronX Functions

1. **renderBefore(widget, target) –** renders elements before the target on the web page.

2. **renderAfter(widget, target) –** renders elements after the target.

3. **renderFirstPlace(widget, target) –** renders elements as the first child of the target.

4. **renderLastPlace(widget, target) –** renders elements as the last child of the target.

5. **pushBefore(target, "contents") –** pushes HTML contents as the first content of the target.

6. **pushAfter(target, "contents")** - pushes HTML contents as the last content of the target.

7. **removeContent(target, "contents") –** searches and removes contents from the target.

8. **replaceContent(target, "existing contents", "new contents") –** searches and replaces contents in the target with new contents.

9. **removeContents (target, "contents") –** searches and removes all same contents at the same time.

10. **replaceContents(target, "existing contents", "new contents") –** searches and replaces all same contents at once with a new one.

11. **insertBefore(target, "str", "new content") –** searches and adds new contents before a string on the web page.

12. **insertAfter(target, "str", "new content")** – searches and adds new contents before a string on the web page.

13. **insertBeforeAll(target, "str", "new content") –** searches and adds new contents before all same string on the webpage.

14. **insertAfterAll(target, "str", "new content") –** searches and adds new contents before all same string on the webpage.

Widget Cloning

15. **clone(target, widget) –** used to clone a widget into a target.

16. **cloneRepeat(target, widget) –** This lets you clone the same widget into a group of targets.

17. **clones(target, widget, n) –** This lets you drop multiple clones of a single widget into a single target.

(This also applies to cloning before, after, within in first place/last place of any existing widget/target/element. For instance – cloneRepeatLastPlace(target, widget) & clonesAfter(target, widget, n).

Content Manipulation

Contenting

18. **swap(widget 1, widget 2) –** This lets you swap widgets.

19. **html(target, content) –** Lets you return HTML contents of a widget (using **html(widget)**) and even lets you rewrite them (using **html(widget, new contents)**).

20. **text(target, content) –** Lets you return text contents of a widget (using **text(widget)**) and even lets you rewrite them (using **text(widget, new text)**).

21. **conText(target, content) –** Lets you return text contents of a widget (using **conText(widget)**) and even lets you rewrite them (using **conText(widget, new text)**).

22. **val(target, content) –** Lets you return the value of a widget (using **val(widget)**) and even lets you rewrite them (using **val(widget, new value), mostly used for form elements/data**).

23. **attrib(target, attribute, data) –** Lets you return the attribute data/value of a widget (using **attrib(target, attribute)**) and even lets you rewrite them (using **attrib(target, attribute, data)**).

Styling

22. **css(target, "attribut1 e: value 1; attribute 2: value 2;...") –** Lets you define a series of new styles for a widget.

Functionality

23. **tplclick(target, func) –** This lets you create triple click functionality just like doubleclick in HTML5, then place your statements, just like this **tplclick(target, function () => {statements}.**

24. **restrict(target) –** This function disables a widget.

25. **restrictAll(target) –** This function disables an entire group of widgets.

26. **unrestrict(target)** – This function enables widgets.

27. **unrestrictAll(target) –** This function enables an entire group of widgets.

Animation

28. **animate(group, mode, time, leftbtn, rightbtn, indicators) –** This function places a [group] of widgets in animation mode [mode – automatic/manual], according to the [time] set, with [indicators] to determine the position of the current displayed picture. If the mode is manual, you can add a [leftbtn] and [rightbtn].

Visibility State

29. **hide(widget) –** Lets you hide a widget.

30. **hideAll(widgets) –** Lets you hide a group of widgets.

31. **unhide(widgets) –** Lets you unhide a widget.

32. **unhideAll(widgets) –** Lets you unhide a group of widgets.

33. **del(widget) –** Lets you delete a widget.

34. **delAll(widget) –** Lets you delete a group of widgets.

Iteration

35. **randomize(widget) –** Lets you display random data or widgets from a group of data or widgets.

36. **randomSelect(group, num, func) –** This function lets you select random set of data according to a [num] from a [group] and returns them as i.

37. **randomsOfRange(start, end, num, func) –** This function lets you pick a random set according to the specified range ([start] & [end]) and [num] of random data, returning them as i.

38. **loop(group, func) –** This function, just like the for loop lets you iterate groups.

39. **loopX(group, func) –** With this function, "I" becomes "group[i]" and is used instead of "group[i]".

40. **rangeSelect(start, end, func) –** This lets you select/iterate a group within a specific range, and returns the value of i.

41. **singleSelect([group/array], func) –** This lets you manually iterate and select from a group.

42. **loopSelect(group, num/position, func) –** This lets you repeatedly select a [num/position] multiplied by 1++ continuously within the group, then returns i.

43. **typeSelect(group, type, func) –** This lets you iterate a group by [type] "even" or "odd".

Integers

44. **randomOf(num, func) –** This returns a random integer/position from [num], as n.

45. **randomofN(num) –** This returns n as a random number from [num[.

46. **randomOfRange(min, max) –** This returns n as a random number within a specific range [min] & [max].

47. **arrayOf(num, func) –** This returns as an array of integers/positions [num], as n.

48. **arrayOfN(num) –** This returns n as an array of numbers (1 to [num]).

49. **rangeOfN(min, max) –** This returns an range of numbers ([min] to [max]).

Shuffle, Remix & Typewrite

50. **shuffle(text/content) & remix(text/content) –** These functions, performing the same task, lets you scatter or rearrange data, text or contents.

51. **shuffleWithin(group, text/content) –** This function lets you shuffle and render [text/content] into a group of elements/targets/widget.

52. **typewrite(target, text, arrayofcolors, loop[“true”/”false”], time, direction[null/”reverse”]) –** This function lets you retype text with target, with an mixed [arrayofcolors] for styling, repeatedly if set as [“true”} or {“false”], according to the [time] set at an interval for controlling the speed, and the direction [reverse] to make it count backwards.

Importing An Array Of Stylesheets

53. **importStylesheet(source) –** This function lets you import a stylesheet.

54. **importStylesheets(array_of_sources) –** This function lets you import an array of stylesheets.

Tracing & Untracing Elements

55. **trace.on(widget) –** Lets you trace and add border to the child elements of a widget.

56. **trace.off(widget) -** Lets you untrace and remove border from the child elements of a widget.

Switch between Contents

57. **switchContent(class, id) –** This, being the only SpectronX content manipulation function that does not require you to select a widget, lets you switch between contents of a web app, hiding them by their class name and displaying each one by their id.

Popping Widgets

58. **popup(widget) –** Lets you pop a widget to the screen.

59. **popdown(widget) –** Lets you remove the popup.

CSS Transform Functions

The following functions are identical (has same functionalities) to the CSS 3 transform properties.

60. **rotate(widget, value)**

61. **rotateX(widget, value)**

62. **rotateY(widget, value)**

63. **rotateZ(target, value)**

64. **translate(target, valueX, valueY)**

65. **translateX(target, value)**

66. **translateY(target, value)**

67. **scale(target, value)**

68. **scaleX(target, value)**

69. **scaleY(target, value)**

70. **skew(target, value)**

71. **skewX(target, value)**

72. **skewY(target, value)**

Binding data to Iframe

73. **bindToFrame(target, obj) –** This function lets you bind an object [obj] or object data to an iframe.

Manipulating Classes

74. **removeClass(target, value) –** Lets you remove a class.

75. **addClass(target, value) –** Lets you add a class.

76. **toggleClass(target, value) –** Lets you toggle a class.

77. **replaceClass(target, existingClass, newClass) –** Lets you replace an existing class with a new class.

78. **removeClasses(target, array) –** Lets you remove an array of classes.

79. **addClasses(target, array) –** Lets you add an array of classes.

80. **toggleClasses(target, array) –** Lets you toggle an array of classes.

81. **replaceClasses(target, arrayOfExistingClasses, arrayOfNewClasses) –** Lets you replace an array of existing classes with new classes.

Audio Callbacks (For Chrome and Edge Browsers)

82. **voice.output(data)** – For audio callbacks. Supported majorly by Chrome & Edge browsers.

Events Handling

SpectronX lets you add event listeners by calling them as functions and adding parenthesis within them. For example;

```
Use click($('#id'), () => {
        alert('helloworld!');
});  instead of document.getelementbyId('id').addEventListener('click',
        function () { alert('hello world!');
});
```

Same thing goes for every other existing events.

Objects

SpectronX Objects, also called Combined ROM Components are very interesting predesigned/readymade widgets/objects that are used to support your frontend application development. These include:

83. **loadApp() –** This function is used to create a loader for your app. It binds your app logo and theme color to a container and displays it when a person visits your website/web page/web app.

84. **loadContent(func) –** This function is used to load specific contents. It's was created majorly for loading contents when a link or button is pressed, and you were to call other functions after a link was clicked.

85. **changeAppMode('dark'/''light') –** This function lets you switch your website's theme between ["dark"] and ["light"] mode.

ACHIEVEMENTS

From designing and manipulating the visible aspects of a website/web apps to building web-based games, SpectronX has been used to make web coding and web frontend project management easier and faster.

THE AUTHOR
Ronaldo Excellent

Social Contacts:

Tel: +234 9114349749

Facebook – https://facebook.com/ronaldinhoexcellent

Messenger – https://m.me/ronaldinhoexcellent

Instagram – @ronaldo.excellent

Twitter - @ronaldoexcel

Github – https://github.com/ronaldoexcellent

Reddit – https://reddit.com/user/ronaldoexcellent

Buy Me A Coffee – https://buymeacoffee.com/ronaldoexcel